

typed-korean

덥다 는 어떻게 더워요 가 되는가

한국어 활용 규칙을 TypeScript 타입 시스템만으로



- Twitter: <https://x.com/JoonNot>
- GitHub: <https://github.com/notJoon>
- 프로젝트: <https://github.com/notJoon/typed-korean>

결과부터

```
const wrong: Conjugate<답다, "해요체"> = "답어요";  
//      ^ error TS2322: Type '"답어요"' is not assignable to type '"더워요"'.  
  
const right: Conjugate<답다, "해요체"> = "더워요"; // OK
```

컴파일러는 정답이 "더워요"라는 걸 어떻게 알았을까?

목표와 제약

- 런타임 코드 없이 타입 검사만 활용
- 컴파일 시점의 문자열 리터럴만 다룬다
- 실용적인 목적 X

typed-japanese (<https://github.com/typedgrammar/typed-japanese>)

```
type やる = GodanVerb & { stem: "や"; ending: "る" };  
type 春日影 = ProperNoun<"春日影">;
```

'なんで春日影やったの' is declared but never used. ts(6196)

```
type なんで春日影やったの = "なぜ春日影やったの" | "なんで春日影やったの" | "どうして春日影やったの"
```

Quick Fix... (⌘.)

```
type なんで春日影やったの = InterrogativePhrase<  
  WhyInterrogative,  
  春日影,  
  やる,  
  "た形",  
  "の"  
>;
```

타입 시스템의 핵심 도구

```
// 1. 문자열 만들기
type Greet<N extends string> = `FUR:RAID is ${N}`;
type G = Greet<"fun">; // "FUR:RAID is fun"

// 2. 분기와 패턴 매칭
type Last<S> = S extends `${string}${infer C}` ? C : never;
type L = Last<"답다">; // "다"

// 3. 표 조회
type Table = { _1: "ㄱ"; _2: "나" };
type R = Table["_1"]; // "ㄱ"
```

→ 이후 나오는 코드는 전부 조립 · 분기 · 패턴 매칭 · 조회의 조합

한국어 활용 규칙, 한 장으로

답다

- └ 어간: 답
- └ 모음 어미 앞에서: 더우
- └ 더우 + 어요
- └ ㅌ + ㅍ = ㅍ
- └ 더워요

- 받침 여부에 따라 결합 방식이 달라진다
먹 + 어요 → 먹어요 / 가 + 아요 → 가요
- 마지막 모음에 따라 아/어를 고른다
- 모음끼리 만나면 축약된다
주 + 어요 → 줘요
- 일부 단어는 모음 어미 앞에서 어간이 바뀐다

첫 번째 벽: 코드포인트를 직접 계산할 수 없다



```
type D = DecomposeLastChar<"먹">; // { 초: "ㅁ"; 중: "ㅣ"; 종: "ㄱ" }
type C = Compose<"ㅞ", "ㅏ", "ㅞ">; // "봤"
```

음절 분해: 테이블 역방향 검색으로

```
// 생성된 테이블 (일부)
type ChoTable = { "ㄱ": "가각각갸간..."; "ㄴ": "나낙냔..."; ... };
type JongTable = { "NULL": "가개가..."; "ㄱ": "각객각..."; ... };

// "이 글자를 포함한 키는?" → 매핑된 타입으로 검색
type FindContainingKey<T, C extends string> = {
  [K in keyof T]: T[K] extends `${string}${C}${string}` ? K : never;
}[keyof T];
```

- 분해 테이블: 자모별 긴 문자열 하나에 11,172자 전체
- 조합 테이블: 받침을 ㄴ · ㄹ · ㅂ · ㅅ · 없음으로 제한해 1,995개

참고: <https://hackers.pub/@joonnot/2026/typed-korean-jamo-table-inversion>

활용 파이프라인

동사 + 어미
↓
실제로 사용할 어간 선택 ← ㅂ·ㄷ 불규칙은 여기서 끝
↓
마지막 음절 분해 (DecomposeLastChar)
↓
모음조화 · 축약 규칙 조회 (ContractionTable)
↓
음절 재조합 (Compose)
↓
"더워요"

핵심 설계: 불규칙을 계산하지 않고 저장한다

```
interface IrregularVerb<Type extends IrregularType, Alt extends string = string>
    extends Verb {
        irregularType: Type;
        altStem: Alt;           // 모음 어미 앞에서 쓸 어간
    }

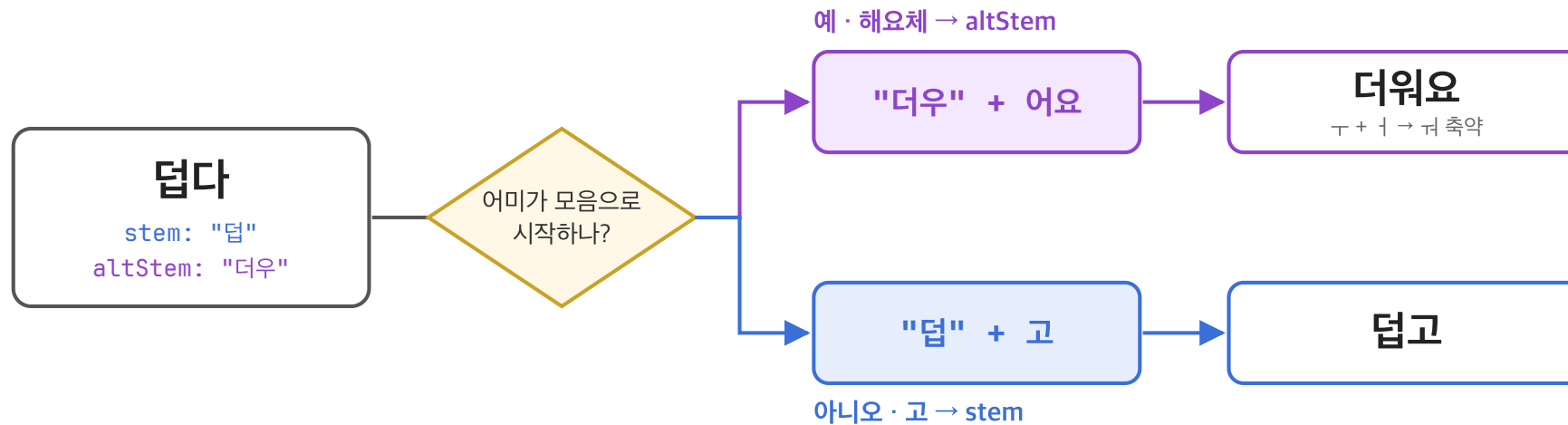
type 덩다 = Adjective & IrregularVerb<"ㅂ", "더우"> & { stem: "덩" };
```

단어	기본 어간 stem	대체 어간 altStem
덩다	덩	더우
듣다	듣	들
모르다	모르	몰

- 엔진은 ㅂ → 우 변환을 계산하지 않는다
- 어휘가 어간 두 개를 주고, 엔진은 어느 것을 쓸지만 결정한다

EffectiveStem: 어미가 어간을 고른다

```
type EffectiveStem<V extends Verb, F extends EndingType> =  
  EndingRuleMap[F]["start"] extends "vowel"  
    ? V extends IrregularVerb<IrregularType> ? V["altStem"] : V["stem"]  
    : V["stem"];
```



→ ㅂ·ㄷ 불규칙의 어간 교체는 이 갈림길 한 곳에 격리된다

따라가 보기: Conjugate<답다, "해요체">

	단계	결과
①	어미 확인	"해요체" → 모음 어미
②	어간 선택	"답" → "더우"
③	마지막 음절 분해	우 → ㅇ + ㅜ
④	모음조화 · 축약	ㅜ + ㅏ → ㅑ
⑤	재조합	더 + 워 + 요 → "더워요"

답다 의 불규칙이 관여하는 것은

어간 선택에서만

이후 부터는 규칙 동사 주다 → 줘요 와 같은 경로

높임 - (으)시- 처리

```
type H = Honorific<덥다>["stem"];  
// "더우시" ← 새로운 규칙 어간  
  
type R = Conjugate<Honorific<덥다>, "과거_평서">;  
// "더우셨다"
```

- 높임 어간을 만든 뒤 기존 활용 엔진에 다시 넣었더니 모든 어미가 동작
- 새 규칙 테이블 추가 X

물론 모든 예외가 altStem 하나로 끝나지는 않는다 (ㅅ · 르 · 러 · 흥 → 부록)

한계와 교훈

1. 계산하기 어려우면 데이터로 바꾼다

유니코드 산술 → 생성된 테이블

2. 예외 처리를 경계에 모은다

바·ㄷ 불규칙의 어간 교체 → 어휘의 `altStem` + `EffectiveStem`

3. 결과를 다시 입력할 수 있게 설계한다

높임 어간 → 기존 활용 파이프라인

한계: 등록된 어휘와 컴파일 시점 리터럴만 처리 가능.

감사합니다

Q & A

<https://github.com/notJoon/typed-korean>

부록

부록 A: 남은 예외들

유형	남은 문제	처리
ㅅ	지 + 어 인데 ㅣ + ㅅ = ㅅ 축약이 일어나면 안 됨 ("저요" X)	축약 건너뛰기 분기
르	모음조화를 몰 이 아니라 원래 어간 모르 의 앞 음절로 판단	르아어Vowel + 전용 vowel base
러	항상 러	전용 vowel base
ㅎ	-(으)면 에서 대체 어간이 아닌 그러면	DropFinalJong
하다	접두 명사를 다시 붙여야 함	전용 테이블 하다ConjugationMap

```
type VerbClass<V> = V extends 하다Verb ? "하다"
  : V extends IrregularVerb<"르"> ? "르"
  : V extends IrregularVerb<"러"> ? "러" : "default";
```

부록 B: 조사와 문장 구조



- 조사: [받침0, 받침X] 튜플 + 받침 분기 한 곳 (밥을 , 나는)
- 문장: 역할 슬롯을 보존한 구조체 → 의문사가 들어갈 슬롯이 이미 차 있으면 에러

부록 C: 테스트와 성능

```
type _모르다 = AssertAll<ConjugateTest<모르다, [  
  ["해요체", "몰라요"],  
  ["과거_평서", "몰랐다"],  
  ["고", "모르고"],  
// 틀리면 → { error: "Type mismatch"; actual: ...; expected: ... }
```

- `tsc --noEmit` 이 곧 테스트 러너
- 함정: 제네릭 별칭 내부의 제약은 인스턴스화 시점에 다시 검사되지 않는다 → `AssertAll` 은 호출부에서
- 전체 타입 검사 약 0.9초 (check 0.28초, 인스턴스화 약 87,300회, 어휘 31개)